Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 33

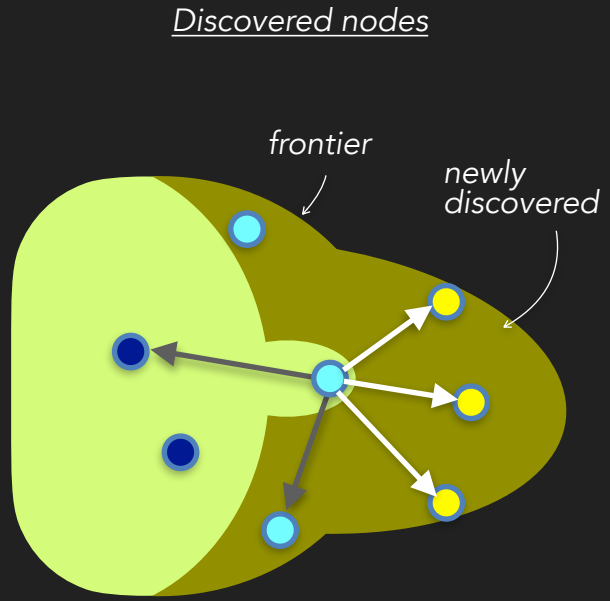
Graphs

Breadth-First Search

Exploring a Graph

- Recall the template for graph exploration:

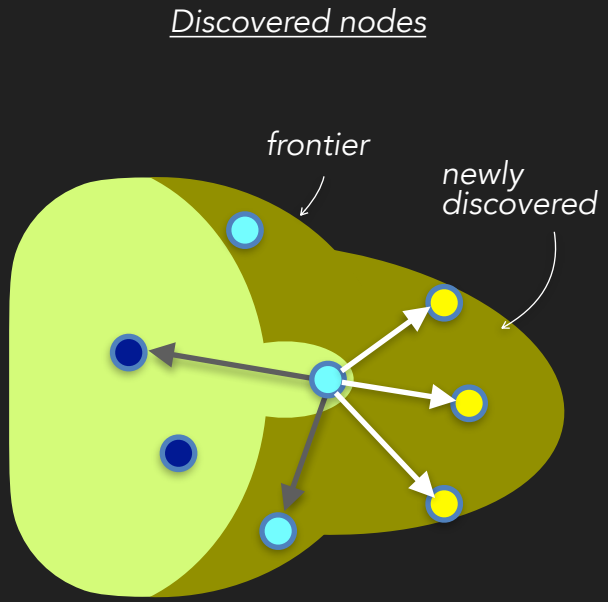
```
// discovered[v] false for all nodes v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    frontier.insert(v); discovered[v] = true;
  }
}
```



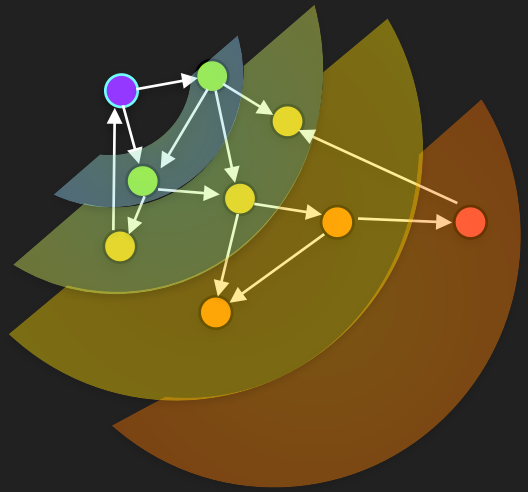
- With any container (which guarantees that every inserted item will be extracted before turning empty) will reach all reachable nodes

Exploring a Graph

- Time?
  - Each reachable node gets into and out of the frontier exactly once. ≤ min(|V|,|E|) reachable nodes.
    - ≤ min(|V|,|E|) iterations of the loop
  - In each iteration, examine each incident edge (inner loop). Each reachable edge examined once: O(|E|) time examining edges
    - If using adjacency matrix, |V| time to discover incident edges in each iteration
  - Total time: O(|E|) or $O(|V|^2)$ if using adjacency matrix



Breadth-First Search (BFS)

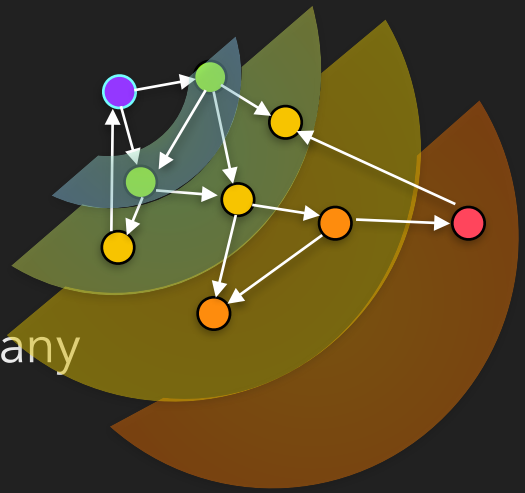


- BFS if frontier implemented as a FIFO structure (queue)
- To study BFS, we first define *level sets* in a graph
- Let d(s,v), for *distance* from s to v, denote the shortest length of a path from s to v (with d(s,s)=0).
- *Level-set* $L_{s,k} = \{ v \mid d(s,v) = k \}$ (when s understood, will call it simply L_k)

1. All edges reachable from s are from $L_{s,j}$ to $L_{s,k}$ where $k \leq j+1$ **[Why?]**
2. If $v \in L_k$ $(k>0)$, there is an edge (in any shortest s-v path) (b,v) s.t. $b \in L_{k-1}$ **[Why?]**

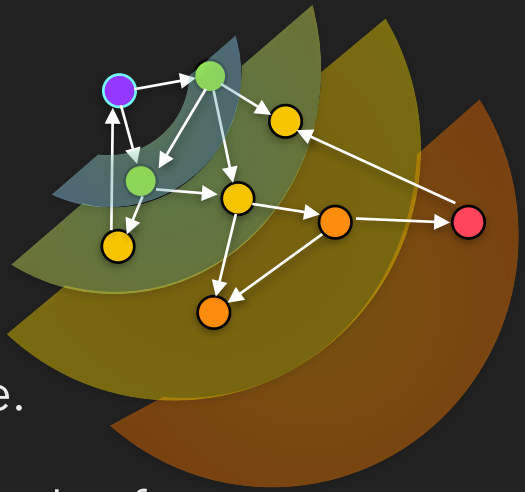
- Claim: In BFS, nodes enter the frontier queue ordered by levels

Breadth-First Search (BFS)



- Claim: For all j, all reachable nodes in L_j are enqueued before any in node in $L_{>j}$ is enqueued
- Base case: j=0: only s is in L_0 and it is enqueued first
- Suppose holds for up to j=k-1 (k>0). To prove that for any node $v \in L_k$ no node in $L_{>k}$ enqueued before it.
  - Let z be the first node in $L_{>k}$ enqueued, discovered through edge (x,z).
  - Choice of $z \Rightarrow x \in L_{\leq k}$, $z \in L_{>k}$. Since (x,z) is an edge, by 1, $x \in L_k$, $z \in L_{k+1}$
  - By 2, there is a node $u \in L_{k-1}$ with an edge (u,v). By IH, u enqueued before x enqueued. Queue ⇒ u extracted before x extracted.
  - when v enqueued ≤ when u extracted < when x extracted = when z enqueued

Breadth-First Search (BFS)



- Node $v \in L_k$ must be discovered from a node $u \in L_{k-1}$
  - While extracting from the queue, level-by-level, v not seen from any node in $L_{<k-1}$ (1) and is seen from at least one node in L_{k-1} (2): u is first of those.
- Aside: so at any point, the frontier queue has all its nodes from at most two adjacent levels (or has all the reachable nodes from one level)
- The levels of nodes entering the queue can be inductively kept track of
  - If level of the dequeued node = k-1, level of enqueued nodes = k

Attendance Break!

Login using CSE LDAP id at

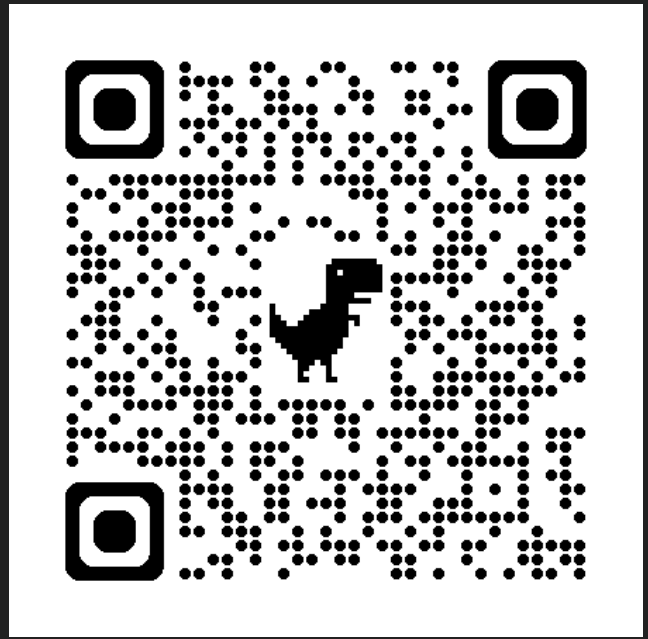
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



BFS and Shortest-Path

- An important application in a graph: Given a pair of source-target nodes (s,t), check if t reachable from s, and if so, find the distance d(s,t)
  - Run BFS from source s and if t is reached, report the level k s.t. $t \in L_{s,k}$
- Can we also find a shortest path from s to t?
  - Yes! When a node v is discovered from u, record `discovered[v]=u` (instead of just a boolean flag); use, say, `-1` to indicate not discovered, and let `discovered[s]=s`
  - Let $u_0 = t$, u_{i+1} = `discovered[`u_i`]`, until $u_k == s$. Path is $u_k, u_{k-1}, \ldots, u_0$
  - In the traversal template, with any container , gives some s-t path